

条款 48: 认识 template 元编程

Be aware of template metaprogramming.

Template metaprogramming (TMP, 模板元编程) 是编写 template-based C++ 程序并执行于编译期的过程。花一分钟想想这个: 所谓 template metaprogram (模板元程序) 是以 C++ 写成、执行于 C++ 编译器内的程序。一旦 TMP 程序结束执行, 其输出, 也就是从 templates 具现出来的若干 C++ 源码, 便会一如往常地被编译。

如果这没有带给你异乎寻常的印象, 你一定没有足够认真地思考它。

C++ 并非是为 template metaprogramming 而设计, 但自从 TMP 于 1990s 初期被发现以后, 由于日渐被证明十分有用, 其延伸部分很可能加入语言和标准程序库内, 使 TMP 更容易进行。是的, TMP 是被发现而不是被发明出来的。当 templates 加入 C++ 时 TMP 底层特性也就被引进了。对某些人而言唯一需要注意的是如何以熟练巧妙而意想不到的方式使用 TMP。

TMP 有两个伟大的效力。第一, 它让某些事情更容易。如果没有它, 那些事情将是困难的, 甚至不可能的。第二, 由于 template metaprograms 执行于 C++ 编译期, 因此可将工作从运行期转移到编译期。这导致的一个结果是, 某些错误原本通常在运行期才能侦测到, 现在可在编译期找出来。另一个结果是, 使用 TMP 的 C++ 程序可能在每一方面都更高效: 较小的可执行文件、较短的运行期、较少的内存需求。然而将工作从运行期移转至编译期的另一个结果是, 编译时间变长了。是的, 程序如果使用 TMP, 其编译时间可能远长于不使用 TMP 的对应版本。

考虑 p.228 导入的 STL advance 伪码(位于条款 47。或许你会想现在就阅读它, 因为本条款中我假设你已经熟悉条款 47 的内容)。就像 p.228 所示, 我特别强调那段代码的伪码部分 (pseudo part):

```
template<typename IterT, typename DistT>
void advance(IterT& iter, DistT d)
{
    if (iter is a random access iterator) {
        iter += d;    //针对 random access 迭代器使用迭代器算术运算
    }
    else {
        if (d >= 0) { while (d--) ++iter; }    //针对其他迭代器类型
        else { while (d++) --iter; }          //反复调用 ++ 或 --
    }
}
```

我们可以使用 `typeid` 让其中的伪码成真，取得 C++ 对此问题的一个“正常”解决方案——所有工作都在运行期进行：

```
template<typename IterT, typename DistT>
void advance(IterT& iter, DistT d)
{
    if (typeid(typename std::iterator_traits<IterT>::iterator_category)
        == typeid(std::random_access_iterator_tag)) {
        iter += d;           //针对 random access 迭代器，使用迭代器算术运算。
    }
    else {
        if (d >= 0) { while (d--) ++iter; }    //针对其他迭代器分类
        else { while (d++) --iter; }          //反复调用 ++ 或 --
    }
}
```

条款 47 指出，这个 `typeid-based` 解法的效率比 `traits` 解法低，因为在此方案中，(1) 类型测试发生于运行期而非编译期，(2) “运行期类型测试”代码会出现在（或说被连接于）可执行文件中。实际上这个例子正可彰显 TMP 如何能够比“正常的”C++ 程序更高效，因为 `traits` 解法就是 TMP。别忘了，`traits` 引发“编译期发生于类型身上的 `if...else` 计算”。

稍早我曾谈到，某些东西在 TMP 比在“正常的”C++ 容易，对此 `advance` 也提供了一个好例子。条款 47 曾经提过 `advance` 的 `typeid-based` 实现方式可能导致编译期问题，下面就是个例子：

```
std::list<int>::iterator iter;
...
advance(iter, 10);    //移动 iter 向前走 10 个元素；
                     //上述实现无法通过编译。
```

下面这一版 `advance` 便是针对上述调用而产生的。将 `template` 参数 `IterT` 和 `DistT` 分别替换为 `iter` 和 `10` 的类型之后，我们得到这些：

```
void advance(std::list<int>::iterator& iter, int d)
{
    if (typeid(std::iterator_traits<std::list<int>::iterator>::iterator_category)
        == typeid(std::random_access_iterator_tag)) {
        iter += d;           //错误！
    }
    else {
        if (d >= 0) { while (d--) ++iter; }
        else      { while (d++) --iter; }
    }
}
```

问题出在我所强调的那一行代码使用了 += 操作符，那便是尝试在一个 `list<int>::iterator` 身上使用 +=，但 `list<int>::iterator` 是 *bidirectional* 迭代器（见条款 47），并不支持 +=。只有 *random access* 迭代器才支持 +=。此刻我们知道绝不会执行起 += 那一行，因为测试 typeid 的那一行总是会因为 `list<int>::iterators` 而失败，但编译器必须确保所有源码都有效，纵使是不会执行起来的代码！而当 iter 不是 *random access* 迭代器时 "iter += d" 无效。与此对比的是 traits-based TMP 解法，其针对不同类型而进行的代码，被拆分为不同的函数，每个函数所使用的操作（操作符）都可施行于该函数所对付的类型。

TMP 已被证明是个“图灵完全”（Turing-complete）机器，意思是它的威力大到足以计算任何事物。使用 TMP 你可以声明变量、执行循环、编写及调用函数……但这般构件相对于“正常的”C++ 对应物看起来很是不同，例如条款 47 展示的 TMP `if...else` 条件句是藉由 templates 和其特化体表现出来。不过那毕竟是汇编语言层级的 TMP。针对 TMP 而设计的程序库（例如 Boost's MPL，见条款 55）提供更高层级的语法——尽管目前还不足以让你误以为那是“正常的”C++。

为了再次浮光掠影地认识一下“事物在 TMP 中如何运作”，让我们看看循环。TMP 并没有真正的循环构件，所以循环效果系藉由递归（recursion）完成。如果你对递归不太适应，恐怕必须在大胆投入 TMP 之前先解决它。TMP 主要是个“函数式语言”（functional language），而递归之于这类语言就像电视之于美国通俗文化一样地无法分割。TMP 的递归甚至不是正常种类，因为 TMP 循环并不涉及递归函数调用，而是涉及“递归模板具现化”（recursive template instantiation）。

TMP 的起手程序是在编译期计算阶乘（factorial）。这不是个令人特别兴奋的程序，但 "hello world" 程序也不是，而两者对于语言的导入都很有帮助。TMP 的阶乘运算示范如何通过“递归模板具现化”（recursive template instantiation）实现循环，以及如何在 TMP 中创建和使用变量：

```
template<unsigned n>                                //一般情况: Factorial<n> 的值是
struct Factorial {                                  // n 乘以 Factorial<n-1> 的值。
    enum { value = n * Factorial<n-1>::value };
};

template<>                                           //特殊情况:
struct Factorial<0> {                               //Factorial<0> 的值是 1
    enum { value = 1 };
};
```

有了这个 `template metaprogram`（其实只是个单一的 `template metafunction` `Factorial`），只要你指涉 `Factorial<n>::value` 就可以得到 `n` 阶乘值。

循环发生在 `template` 具现体 `Factorial<n>` 内部指涉另一个 `template` 具现体 `Factorial<n-1>` 之时。和所有良好递归一样，我们需要一个特殊情况造成递归结束。这里的特殊情况是 `template` 特化体 `Factorial<0>`。

每个 `Factorial` `template` 具现体都是一个 `struct`，每个 `struct` 都使用 `enum hack`（见条款 2）声明一个名为 `value` 的 `TMP` 变量，`value` 用来保存当前计算所得的阶乘值。如果 `TMP` 拥有真正的循环构件，`value` 应该在每次循环内获得更新。但由于 `TMP` 系以“递归模板具现化”（`recursive template instantiation`）取代循环，每个具现体有自己的一份 `value`，而每个 `value` 有其循环内的适当值。

你可以这样使用 `Factorial`：

```
int main()
{
    std::cout << Factorial<5>::value;      //印出 120
    std::cout << Factorial<10>::value;     //印出 3628800
}
```

如果你认为这比冬天吃冰淇淋还酷，你就是取得了成为一个 `template metaprogrammer` 的必要条件。如果 `templates` 和其特化版本，以及递归具现化和 `enum hacks`，以及键入 `Factorial<n-1>::value` 这样的东西会使你汗毛直竖，唔……你是个十分“正常化”的 C++ 程序员。

当然，`Factorial` 示范 `TMP` 的用途就只是像 “hello world” 示范任何传统语言的用途一样。为求领悟 `TMP` 之所以值得学习，很重要的一点是先对它能够达成什么目标有一个比较好的理解。下面举出三个例子：

- 确保量度单位正确。在科学和工程应用程序中，确保量度单位（例如质量、距离、时间……等等）正确结合是绝对必要的。举个例子，将一个质量变量赋值给一个速度变量是错误的，但是将一个距离变量除以一个时间变量并将结果赋值给一个速度变量则成立。如果使用 `TMP`，就可以确保（在编译期）程序中所有量度单位的组合都正确，不论其计算多么复杂。这也就是为什么 `TMP` 可被用来进行早期错误侦测。这种 `TMP` 用途的一个有趣情况是，就连因次为分数的指数（`fractional dimensional exponents`）也可支持，但分数必须先在编译期被约简，

例如 $\text{time}^{1/2}$ 的单位和 $\text{time}^{4/8}$ 的单位相同。

- 优化矩阵运算。条款 21 曾经提过某些函数包括 `operator*` 必须返回新对象，而条款 44 又导入了一个 `SquareMatrix class`。考虑以下代码：

```
typedef SquareMatrix<double, 10000> BigMatrix;  
BigMatrix m1, m2, m3, m4, m5;           //创建矩阵并  
...                                     //赋予它们数值。  
BigMatrix result = m1 * m2 * m3 * m4 * m5; //计算它们的乘积。
```

以“正常的”函数调用动作来计算 `result`，会创建 4 个暂时性矩阵，每一个用来存储对 `operator*` 的调用结果。犹有进者，各自独立的乘法产生了 4 个作用于矩阵元素身上的循环。如果使用高级、与 TMP 相关的 `template` 技术，即所谓 *expression templates*，就有可能消除那些临时对象并合并循环，这一切都无需改变客户端的用法（像上面那样）。于是 TMP 软件使用较少的内存，执行速度又有戏剧性的提升。

- 可以生成客户定制之设计模式（*custom design pattern*）实现品。设计模式如 **Strategy**（见条款 35），**Observer**，**Visitor** 等等都可以多种方式实现出来。运用所谓 *policy-based design* 之 TMP-based 技术，有可能产生一些 `templates` 用来表述独立的设计选项（所谓 “*policies*”），然后可以任意结合它们，导致模式实现品带着客户定制的行为。这项技术已被用来让若干 `templates` 实现出智能指针的行为政策（*behavioral policies*），用以在编译期间生成数以百计不同的智能指针类型。这项技术已经超越编程工艺领域如设计模式和智能指针，更广义地成为 *generative programming*（殖生式编程）的一个基础。

不是每个人都喜欢 TMP。其语法不直观，其支持工具目前还不充分（`template metaprograms` 的调试器？哈，还早咧！）由于 TMP 是一个在相对短时间之前才意外发现的语言，其编程方式还多少需要倚赖经验。尽管如此，将工作从运行期移往编译期所带来的效率改善还是令人印象深刻，而 TMP 对“难以或甚至不可能于运行期实现出来的行为”的表现能力也很吸引人。

TMP 仿佛旭日东升。有可能下一版 C++ 会对它提供明确的支持，甚至 TR1 已经这样做了（见条款 54）。TMP 书籍已逐渐出现，网络上的 TMP 信息愈来愈丰富。

TMP 或许永远不会成为主流,但对某些程序员——特别是程序库开发人员——几乎确定会成为他们的主要粮食。

请记住

- **Template metaprogramming** (TMP, 模板元编程) 可将工作由运行期移往编译期,因而得以实现早期错误侦测和更高的执行效率。
- **TMP** 可被用来生成“基于政策选择组合” (**based on combinations of policy choices**) 的客户定制代码,也可用来避免生成对某些特殊类型并不适合的代码。

8

定制 new 和 delete

Customizing new and delete

当计算环境（例如 Java 和 .NET）夸耀自己内置“垃圾回收能力”的当今，C++ 对内存管理的纯手工法也许看起来有点老气。但是许多苛刻的系统程序开发人员之所以选择 C++，就是因为它允许他们手工管理内存。这样的开发人员研究并学习他们的软件使用内存的行为特征，然后修改分配和归还工作，以求获得其所建置的系统最佳效率（包括时间和空间）。

这样做的前提是，了解 C++ 内存管理例程的行为。这正是本章焦点。这场游戏的两个主角是分配例程和归还例程（allocation and deallocation routines，也就是 operator new 和 operator delete），配角是 new-handler，这是当 operator new 无法满足客户的内存需求时所调用的函数。

多线程环境下的内存管理，遭受单线程系统不曾有过的挑战。由于 heap 是一个可被改动的全局性资源，因此多线程系统充斥着发狂访问这一类资源的 race conditions（竞速状态）出现机会。本章多个条款提及使用可改动之 static 数据，这总是会令线程感知（thread-aware）程序员高度警戒如坐针毡。如果没有适当的同步控制（synchronization），一旦使用无锁（lock-free）算法或精心防止并发访问（concurrent access）时，调用内存例程可能很容易导致管理 heap 的数据结构内容败坏。我不想一再提醒你这些危险，我只打算在这里提一下，然后假设你会牢记在心。

另外要记住的是，operator new 和 operator delete 只适合用来分配单一对象。Arrays 所用的内存由 operator new[] 分配出来，并由 operator delete[] 归还（注意两个函数名称中的 []）。除非特别表示，我所写的每一件关于 operator new 和 operator delete 的事也都适用于 operator new[] 和 operator delete[]。